Lecture 27.08: Multi-Model Training & Evaluation — Baseline Classification

📌 Executive Summary

Training and benchmarking Logistic Regression, K-Nearest Neighbors (k=5), and Gaussian Naive Bayes, evaluating Confusion Matrix, Precision, Recall, and F1-Score.

🏛️ Theoretical Formulation & Mathematical Architecture

🥊 Multi-Model Architectural Benchmark Suite

To discover the optimal inductive bias for credit underwriting, we establish three distinct mathematical learners on the standardized training partition $\mathcal{D}_{exttrain}$:

1. Discriminative Linear Hyperplane: Logistic Regression

Models the posterior probability via the Sigmoid logistic link:

$$P(Y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = rac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

Loss function is convex Binary Cross-Entropy (Log-Loss):

$$J(\mathbf{w}, b) = -rac{1}{m} \sum_{i=1}^m \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) ight]$$

2. Non-Parametric Metric Space: K-Nearest Neighbors ($k = 5$)

Assigns class probability via local majority density within neighborhood $\mathcal{N}_k(\mathbf{x})$:

$$P(Y = 1 | \mathbf{x}) = rac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i$$

3. Generative Class-Conditional Density: Gaussian Naive Bayes

Applies Bayes' rule under the conditional feature independence assumption:

$$P(Y = c \mid \mathbf{x}) \propto P(Y = c) \prod_{j=1}^{d} \frac{1}{\sqrt{2\pi \sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \right)$$

📊 Comparative Baseline Metrics Matrix

Classifier	Accuracy	Precision (Safety)	Recall (Coverage)	F1-Score	Primary Architectural Characteristic
Logistic Regression	$\approx 84.7\%$	$\approx 78.5\%$	$\approx 71.4\%$	≈ 0.75	Linear decision boundary, high interpretability
KNN ($k = 5$)	$\approx 81.6\%$	$\approx 73.1\%$	$\approx 64.3\%$	≈ 0.68	Non-linear local tessellation, sensitive to local noise
Gaussian Naive Bayes	$\approx 83.2\%$	$\approx \mathbf{88.6\%}$	$\approx 55.4\%$	≈ 0.68	**Maximum Precision**, lowest false approval defaults

---

💻 Vectorized Python Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_scor

models = {
    "Logistic Regression": LogisticRegression(),
    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Gaussian Naive Bayes": GaussianNB()
}

for name, clf in models.items():
    clf.fit(X_train_scaled, y_train)
    preds = clf.predict(X_test_scaled)
```

```
    print(f"=== {name} ===")
    print("Precision:", round(precision_score(y_test, preds), 4))
    print("Recall:   ", round(recall_score(y_test, preds), 4))
    print("F1-Score: ", round(f1_score(y_test, preds), 4))
    print("Accuracy: ", round(accuracy_score(y_test, preds), 4))
    print("Confusion Matrix:
", confusion_matrix(y_test, preds), "
")
```

🔬 Key Engineering Takeaways & Invariants

1. **Mathematical Grounding**: Every transformation and classification step directly addresses the underlying empirical distribution and financial risk constraints.
2. **Reproducibility**: Clean, leak-free pipeline partitioning guarantees that performance metrics match production operating realities.
3. **Domain Alignment**: Aligning model evaluation with the asymmetric cost of financial errors protects institutional solvency.